

EXPERT EN DÉVELOPPEMENT LOGICIEL

RNCP 39583 - Niveau 7

Bloc 2

Concevoir et développer des applications logicielles

Projet support : YALLO

Auteur	Lenny Copeaux
Formation	YNOV Campus
Date	Juillet 2026
Version de YALLO	2
Dépôt	Application SaaS déployée sur app.yallo.fr

Compétences du référentiel couvertes

Code	Compétence	Réalisation dans Yallo
C2.1.1	Environnements de déploiement et de test	3 environnements (local, staging, production) + SonarCloud, Sentry, Codecov
C2.1.2	Intégration continue	Pipelines GitHub Actions ci.yml et staging.yml
C2.2.1	Prototype applicatif	MVP web responsive : marketing, dashboard cuisine, admin, agent vocal Vapi
C2.2.2	Harnais de tests unitaires	29 fichiers, ~215 cas de test, couverture 72,55 % des instructions
C2.2.3	Développement sécurisé et accessible	OWASP Top 10 adressé ; WCAG 2.1 AA via Radix UI / shadcn
C2.2.4	Déploiement progressif	Branches develop -> staging -> main -> production
C2.3.1	Cahier de recettes	Scénarios fonctionnels, techniques et sécurité (section 12)
C2.3.2	Plan de correction des bogues	7 anomalies tracées, série webhook agent vocal (section 13)
C2.4.1	Documentation technique	Manuels déploiement, utilisation, mise à jour (sections 14-16)

Table des matières

1. Protocole de déploiement continu
2. Critères de qualité et de performance
3. Protocole d'intégration continue
4. Architecture logicielle structurée
5. Présentation d'un prototype réalisé
6. Frameworks et paradigmes de développement
7. Jeu de tests unitaires
8. Mesures de sécurité
9. Actions pour l'accessibilité
10. Historique des versions
11. Présentation de la dernière version
12. Cahier de recettes
13. Plan de correction des bogues
14. Manuel de déploiement
15. Manuel d'utilisation
16. Manuel de mise à jour

Contexte du projet

Yallo est un SaaS vertical pour la restauration rapide. Il permet à un restaurant indépendant (fast-food) de recevoir des commandes téléphoniques via un **agent vocal IA** (Vapi), de les visualiser en temps réel dans un **dashboard cuisine** (tablette) et de les synchroniser optionnellement avec son logiciel de caisse (HubRise).

Problématique : Les restaurants reçoivent encore une part significative de leurs commandes par téléphone sans outil numérique adapté. Les erreurs de prise de commande et l'absence de visibilité en cuisine dégradent le service client.

Solution : 4 surfaces complémentaires :

- **Site marketing** (yallo.fr) - Conversion et information
- **Dashboard restaurant** (app.yallo.fr/dashboard) - Suivi des commandes en cuisine
- **Panel admin** (app.yallo.fr/admin) - Onboarding et gestion plateforme
- **Agent vocal Vapi** - Prise de commande automatisée par téléphone

Le choix d'une architecture monolithique modulaire (Next.js full-stack) a été choisi pour permettre d'itérer rapidement tout en conservant une base industrialisable (avec des tests, une CI/CD et des migrations versionnées).

1. Protocole de déploiement continu

1.1 Objectif

Le déploiement continu (CD) livre chaque version validée sur Vercel sans intervention manuelle avec traçabilité de type (commit -> build -> URL de production).

1.2 Environnements

Environnement	URL	Base de données	Déclencheur
Local	localhost:3000 / app.localhost:3000	Supabase Postgres (branche dev)	pnpm dev
Staging	staging.yallo.fr, app.staging.yallo.fr	Supabase Postgres staging	Push sur develop
Production	yallo.fr, app.yallo.fr	Supabase Postgres production	Push sur main

La séparation repose sur des variables d'environnement Vercel. NEXT_PUBLIC_APP_URL est injectée au build ainsi que le middleware multi-domaine (middleware.ts) qui isole le marketing du domaine applicatif.

1.3 Séquence de déploiement production

Fichier : .github/workflows/ci.yml

```
Push sur main
-> Lint (ESLint + tsc --noEmit)
-> Test (Vitest run + Couverture = Codecov)
-> Sonarcloud (Analyse statique + lcov.info)
-> Build (next build + 1 artefact .next)
-> Job deploy-production (vercel deploy --prod)
-> Security (pnpm audit en parallèle + informatif)
```

Étapes du job deploy-production :

1. Checkout du commit SHA
2. Installation Vercel CLI
3. `vercel deploy --prod --token=$VERCEL_TOKEN` avec secrets `VERCEL_ORG_ID` et `VERCEL_PROJECT_ID`

Condition : `github.ref == refs/heads/main && github.event_name == push`.

1.4 Séquence staging

Fichier : `.github/workflows/staging.yml`

Push sur develop

-> Validate (Lint + test:run + build URL staging)

-> Sonarcloud

-> Job deploy-staging (Vercel pull -> vercel build -> vercel deploy --prebuilt)

1.5 Stratégie de branches

Branche	Rôle	CD
feature/*	Développement isolé	Aucun
develop	Intégration pré-production	Staging automatique
main	Référence production	Production automatique

Cycle : Feature -> PR develop -> Validation staging -> PR main -> Production.

1.6 Outils mobilisés

Outil	Rôle
Git / GitHub	Versioning, PR, secrets CI
pnpm 9	Dépendances (pnpm-lock.yaml)
Node.js 20	Runtime build et CI
Vercel	Hébergement, CDN, Edge, serverless
Supabase Postgres	PostgreSQL serverless
Codecov	Historique couverture de tests

1.7 Point d'attention

ESLint et pnpm audit sont configurés avec `continue-on-error: true` dans le `ci.yml`. Ça permet donc d'informer sans bloquer systématiquement le déploiement. Les garde-fous qui eux sont bloquants sont `tsc --noEmit`, les tests Vitest et le build Next.js.

2. Critères de qualité et de performance

2.1 Critères mesurables

Critère	Seuil cible	Mesure actuelle	Outil	Bloquant CI
Couverture instructions	≥ 70 %	72,55 % (341/470)	Vitest v8	Non
Couverture branches	≥ 65 %	67,54 % (231/342)	Vitest v8	Non
Couverture fonctions	≥ 70 %	75 % (57/76)	Vitest v8	Non
Typage strict	0 erreur	strict: true	tsc --noEmit	Oui
Tests unitaires	100 % passants	~215 cas / 29 fichiers	pnpm test:run	Oui
Build production	Succès	next build	CI	Oui
Dette technique	Quality Gate Passed	SonarCloud	-	Non
Erreurs production	Taux stable	Sentry (sendDefaultPii: false)	3 runtimes	N/A

Source : `coverage/lcov.info` qui est généré par `pnpm run test:coverage`.

2.2 Performance applicative

Architecture serverless Vercel :

- **Edge** : Middleware d'authentification (middleware.ts) - latence minimale
- **Serverless Node 20** : Server Actions et routes API (runtime = "nodejs")
- **CDN** : Pages marketing pré-rendues + assets statiques

Optimisations Next.js 16 :

- Server Components par défaut (moins de JavaScript client)
- `revalidatePath()` ciblé après mutations
- JSONB PostgreSQL pour `menuData` et `statusSettings` (ça évite des migrations fréquentes)

2.3 Qualité de code

SonarCloud analyse à chaque push la complexité, la duplication, le code smells et la détection de secrets hardcodés.

Dossier source : `src/`

Dossier tests : `src/__tests__`

Rapport : `coverage/`[lcov.info](#).

ESLint : `eslint-config-next 16` - règles `next/core-web-vitals` et `next/typescript`.

3. Protocole d'intégration continue

3.1 Déclencheurs

Pipeline	Fichier	Événements
CI production	ci.yml	Push et pull_request sur main
CI staging	staging.yml	Push sur develop

3.2 Graphe des jobs - ci.yml

```
lint -----> test -----> sonarcloud
|
|-----> build -----> deploy-production (push main uniquement)
security (parallèle, indépendant)
```

3.3 Détail des séquences

Job lint : Checkout -> Node 20 + pnpm 9 (cache lockfile) -> pnpm install -> pnpm run lint -> npx tsc --noEmit (bloquant).

Job test (needs: lint) : pnpm run test:run (bloquant) -> pnpm run test:coverage -> upload Codecov.

Job sonarcloud (needs: test) : scan SonarSource avec couverture LCOV.

Job build (needs: lint + test) : variables factices (DATABASE_URL, AUTH_SECRET, NEXT_PUBLIC_APP_URL) -> pnpm run build -> artefact .next (7 jours).

Job deploy-production (needs: build) : vercel deploy --prod.

3.4 Configuration des tests

Fichier : vitest.config.ts

- Environnement : jsdom
- Setup : src/__tests__/setup.ts
- Pattern : src/**/*.test.ts, spec.ts, spec.tsx
- Coverage provider : v8, reporters text, json, html, lcov

3.5 Intégration dans le cycle de développement

1. Développement local : pnpm dev + pnpm test:run avant commit
2. PR vers develop -> pipeline staging
3. Validation fonctionnelle sur staging
4. PR vers main -> pipeline complet + déploiement production

Convention de commits : Conventional Commits (feat:, fix:, test:, chore:).

4. Architecture logicielle structurée

4.1 Vue d'ensemble

Application monolithique modulaire Next.js 16, organisée par domaines métier (feature-first).

Présentation : app/(marketing) & (dashboard) & (admin)
components/ui (shadcn/Radix) & components/landing & Server Actions + RSC
Métier (features/) : orders, menu, hours, kitchen-status, billing, settings - actions.ts, validation Zod
Infrastructure (lib/) : auth, rate-limit, services/stripe, vapi-agent, hubrise, twilio-sms, menu-parser avec Drizzle ORM
PostgreSQL (Supabase Postgres) - db/schema.ts

4.2 Organisation des répertoires

Dossier	Responsabilité
src/app/	Routes App Router, layouts, API routes
src/features/	Logique métier par domaine
src/components/	Composants UI réutilisables
src/lib/	Utilitaires transverses, services externes
src/db/	Schéma Drizzle, connexion
drizzle/	Migrations SQL versionnées
src/___tests___/	Tests unitaires
middleware.ts	Routage multi-domaine + contrôle d'accès Edge

4.3 Modèle de données

Rôles utilisateur : ADMIN, OWNER, EMPLOYEE (enum userRole).

Tables principales :

- users - Profil applicatif lié à Supabase Auth (authUserId)
- organizations - Pour les contrats et abonnements Stripe
- organization_members / restaurant_members - Ce sont les multi-tenant
- restaurants - Avec le menu JSONB, les horaires, l'agent Vapi et HubRise
- orders / order_items - Pour les commandes vocales
- call_logs - Pour la traçabilité des appels Vapi

Évolution : 42 fichiers de migration SQL dans drizzle/ appliqués via drizzle-kit.

Le dossier drizzle/ contient des doublons d'index mais cela est sans impact lors du db:push.

4.4 Paradigmes architecturaux

- **Server Actions** ("use server") : mutations sans URL HTTP exposée
- **Validation Zod** : entrée systématiquement validée
- **Guards** (src/lib/auth.ts) : requireAuth(), requireAdmin(), getAccessibleRestaurant()
- **Middleware Edge** : séparation domaines, redirections par rôle (ADMIN -> /admin, EMPLOYEE -> /dashboard, OWNER -> /org)
- **Services encapsulés** (src/lib/services/) : un fichier par intégration externe

4.5 Flux critique - commande vocale

```
Client appelle numéro Twilio
-> Assistant Vapi (createVapiAgent)
-> Tool call submit_order
-> POST /api/vapi/webhook?rid={restaurantId}
    | Vérification header x-vapi-secret (prod stricte)
    | Vérification statut cuisine STOP et horaires
    | Insertion orders + order_items (Drizzle)
    | Push HubRise (non bloquant)
    | SMS confirmation Twilio
-> Dashboard (revalidatePath)
```

4.6 Multi-tenant et gestion des accès

Le modèle organisation -> restaurants -> membres permet à un groupe de gérer plusieurs établissements :

- **OWNER** : membre d'une organization, accès à tous ses restaurants via `organization_members`
- **EMPLOYEE** : accès restreint aux restaurants listés dans `restaurant_members`
- **ADMIN** : accès global plateforme, isolé du dashboard restaurant

Le cookie `yallo_restaurant_id` mémorise le restaurant actif. La fonction `getAccessibleRestaurant()` dans `src/lib/auth.ts` vérifie le droit d'accès avant toute opération métier.

4.7 Routage multi-domaine

Le fichier `middleware.ts` inspecte le header `Host` :

- `yallo.fr` : pages marketing publiques, SEO, pas de session requise
- `app.yallo.fr` : application authentifiée (`/login`, `/dashboard`, `/admin`, `/org`)
- Redirection automatique des routes `app-only` depuis le domaine marketing vers `app.yallo.fr`

5. Présentation d'un prototype réalisé

5.1 Prototype retenu : Dashboard cuisine

Le prototype principal validé est le dashboard restaurant (app.yallo.fr/dashboard), interface opérationnelle pour le personnel en cuisine, conçue pour tablette (grands éléments tactiles, contrastes élevés).

5.2 User stories couvertes

ID	User story	Statut
US-01	En tant que cuisinier, je vois les nouvelles commandes en temps réel	OK
US-02	Je fais progresser une commande (Nouvelle -> En préparation -> Prête -> Livrée)	OK
US-03	Je signale la charge cuisine (Calme / Normal / Rush / Stop)	OK
US-04	L'agent vocal refuse les commandes si statut STOP	OK
US-05	Je configure le menu du restaurant	OK
US-06	Je configure les horaires d'ouverture	OK

5.3 Composants UI fonctionnels

- **Liste des commandes** : cartes avec statut, heure, articles, client
- **Boutons d'action** : changement de statut commande (Server Action updateOrderStatus)
- **Contrôle statut cuisine** : kitchen-status-control.tsx (4 états)
- **Navigation** : sidebar dashboard, sélecteur de restaurant (multi-établissement)

5.4 Spécificités ergonomiques

- Interface responsive (mobile -> tablette -> desktop)
- Palette Tailwind avec contrastes WCAG AA
- Composants Radix UI : focus clavier, dialogues modaux, sélecteurs accessibles
- PWA : dashboard utilisable en mode application sur tablette cuisine

5.5 Exigences de sécurité du prototype

- Accès protégé par middleware + Supabase Auth
- Rôle EMPLOYEE limité aux restaurants assignés (restaurant_members)
- Aucune donnée sensible exposée côté client (clés API serveur uniquement)

5.6 Prototype secondaire : Panel d'administration

Le panel admin (app.yallo.fr/admin) adresse l'onboarding d'un nouveau restaurant :

Écran	Fonctionnalité	Composant
/admin/restaurants	Liste restaurants	restaurants-data-table.tsx
/admin/restaurants/[id]	Fiche détail, agent Vapi	createVapiAgent
/admin/users	Gestion comptes	users-data-table.tsx
/admin/organizations	Gestion organisations	organizations-data-table.tsx

Le flux createVapiAgent (src/app/(admin)/admin/actions.ts) : création assistant Vapi -> import Twilio -> écriture en base -> rollback si échec Twilio.

5.7 Prototype marketing

Le site yallo.fr : accueil, démo, guide, contact, pages légales. Rendu SSR pour le SEO. Formulaire de contact protégé par rate limiting.

Captures : Annexes A (dashboard)

6. Frameworks et paradigmes de développement

6.1 Stack technique

Technologie	Version	Rôle	Justification
Next.js	16.0.7	Framework full-stack	SSR, App Router, Server Actions
TypeScript	5.9	Typage statique	strict: true, contrats sûrs bout-en-bout
React	19.2	Interface utilisateur	Server Components natifs
Tailwind CSS	4	Styles utilitaires	Cohérence visuelle, dark mode
Radix UI / shadcn	latest	Composants accessibles	ARIA natif, WCAG 2.1
PostgreSQL (Supabase)	16	Base relationnelle	JSONB, intégrité référentielle
Drizzle ORM	0.44	ORM typé	SQL paramétré, migrations versionnées
Supabase Auth	2	Authentification	Sessions JWT, pas de mot de passe maison
Vapi	API	Agent vocal IA	Prise de commande téléphonique
Stripe	22	Paieement	Abonnements, webhooks signés
Twilio	API	Téléphonie / SMS	Numéros SDA, confirmation commande
HubRise	API	Intégration caisse POS	Push commandes vers logiciel de caisse
OpenAI GPT-4o	API	Vision	Parsing menu depuis photos
Vitest	4	Tests unitaires	Intégration TypeScript native
Sentry	10	Monitoring	3 runtimes (server, edge, client)

6.2 Paradigmes appliqués

Programmation orientée composants (React) : UI déclarative, composition, Server/Client Components.

Architecture feature-first : chaque domaine métier (orders, menu, hours...) possède ses actions.ts, types et tests associés. Principe de responsabilité unique.

Validation par schéma (Zod) : contrat d'entrée explicite, rejet des données invalides avant accès base.

Inversion de dépendances : les features appellent lib/services/, jamais les APIs externes directement.

Convention over configuration (Next.js App Router) : routing par structure de dossiers, layouts imbriqués, conventions "use server" / "use client".

6.3 Choix structurants justifiés

Server Actions vs REST : réduction de la surface d'attaque (pas d'endpoint HTTP public pour les mutations), typage partagé client/serveur, moins de boilerplate. Les webhooks externes (Vapi, Stripe) restent en routes API classiques.

Drizzle vs Prisma : schéma TypeScript natif (pas de code-gen), SQL paramétré, adapté au serverless.

Supabase Auth vs authentification maison : délégation du hashage, brute-force protection, refresh tokens - réduction du risque OWASP A07.

7. Jeu de tests unitaires

7.1 Infrastructure de test

Élément	Configuration
Framework	Vitest 4 + @testing-library/react
Environnement	jsdom
Setup global	src/ __tests__ /setup.ts
Mocks	Services externes (Stripe, Vapi, HubRise, Twilio, OpenAI)
Couverture	@vitest/coverage-v8 -> lcov.info, coverage-final.json
CI	pnpm run test:run bloquant ; couverture uploadée Codecov + SonarCloud

7.2 Périmètre et volumétrie

Métrique	Valeur
Fichiers de test	29
Cas de test (it)	~215
Couverture instructions	72,55 %
Couverture branches	67,54 %
Couverture fonctions	75 %

7.3 Répartition par module

Module testé	Fichier(s)	Fonctionnalité couverte
Commandes	features/orders/actions.test.ts	CRUD commandes, changement statut
Menu	features/menu/actions.test.ts	Sauvegarde menu, sync Vapi
Horaires	features/hours/actions.test.ts	Validation horaires, sync agent
Statut cuisine	features/kitchen-status/actions.test.ts	4 états, settings JSONB
Facturation	features/billing/usage-actions.test.ts	Usage et abonnement
Admin	app/admin/actions-*.test.ts	Schémas Zod, utilitaires admin
Webhook Vapi	api/vapi/webhook-end-of-call.test.ts	Fin d'appel, call_logs
Webhook Stripe	lib/services/stripe-webhook.test.ts	Signature, sync abonnement
Rate limiting	lib/rate-limit.test.ts	Limite 10 req/60s, reset fenêtre
Auth / org	lib/auth-org.test.ts	Accès multi-restaurant
Validators	lib/validators.test.ts	Schémas Zod métier
Services	lib/services/*.test.ts	HubRise, Twilio, menu-parser, prompts

7.4 Exemple - test du rate limiter

```
// src/___tests___/lib/rate-limit.test.ts
```

```
describe("rateLimit", () => {  
  it("should allow up to MAX_REQUESTS requests", () => {  
    const id = "test-ip:/api/test";  
    for (let i = 0; i < 10; i++) expect(rateLimit(id)).toBe(true);  
    expect(rateLimit(id)).toBe(false); // 11e requête refusée  
  });  
});
```

7.5 Exemple - test webhook Stripe

```
it("should reject invalid stripe signature", () => {  
  expect(() =>  
    stripe.webhooks.constructEvent(payload, "invalid-sig", secret)  
  ).toThrow();  
});
```

Protection OWASP A08 : webhook forgé rejeté avant modification de base.

7.6 Exemple - tests commandes

features/orders/actions.test.ts mockent Drizzle et requireAuth() pour vérifier la progression de statut, le refus d'accès cross-restaurant et l'appel à revalidatePath.

7.7 Stratégie anti-régression

- Tests à chaque push sur main et develop
- Mocks systématiques des services externes
- Cas nominaux et cas d'erreur (payload invalide, STOP, restaurant introuvable)
- Seuil 70 % instructions atteint (72,55 %)
- Composants UI purs : test manuel + Radix pré-validé accessibilité

8. Mesures de sécurité

La sécurité repose sur une défense en profondeur, audité selon le OWASP Top 10 (2021).

#	Catégorie OWASP	Mécanisme Yallo
A01	Broken Access Control	Middleware rôles + requireAuth()/requireAdmin() + ownership restaurant
A02	Cryptographic Failures	Supabase Auth (bcrypt) ; secrets en variables d'env Vercel ; HSTS
A03	Injection	Drizzle ORM paramétré + validation Zod stricte
A04	Insecure Design	Vérification STOP/horaires avant commande ; rollback agent Vapi si Twilio échoue
A05	Security Misconfiguration	6 en-têtes HTTP sécurité ; Sentry sendDefaultPii: false
A06	Vulnerable Components	pnpm audit --audit-level=high en CI ; lockfile versionné
A07	Auth Failures	Supabase JWT + rate limit 10 req/60s sur endpoints sensibles
A08	Integrity Failures	Signature Stripe (constructEvent) ; secret Vapi (x-vapi-secret)
A09	Logging Failures	Sentry 3 runtimes + logger applicatif structuré
A10	SSRF	Aucune URL externe construite depuis input utilisateur

8.1 En-têtes HTTP (next.config.ts)

En-tête	Protection
Strict-Transport-Security	Force HTTPS
X-Frame-Options: SAMEORIGIN	Anti-clickjacking
X-Content-Type-Options: nosniff	Anti MIME-sniffing
Referrer-Policy	Contrôle référent
Permissions-Policy	Désactive caméra, micro, géoloc

8.2 Webhooks

Stripe : vérification stripe-signature via stripe.webhooks.constructEvent().

Vapi : header x-vapi-secret comparé à VAPI_WEBHOOK_SECRET ; en production Vercel, secret obligatoire (401 sinon). Paramètre rid (restaurantId) dans l'URL du webhook.

8.3 Authentification et sessions

Supabase Auth gère credentials, hashage bcrypt, tokens JWT. Migration 0023 a supprimé la colonne password de users. requireAuth() vérifie la session à chaque Server Action.

8.4 Validation Zod

```
const parsed = businessHoursSchema.safeParse(rawData);
if (!parsed.success) return { success: false, error: "Format invalide" };
```

Double protection avec Drizzle paramétré (A03).

8.5 Gestion des erreurs

Erreurs techniques loguées (Sentry) ; message générique côté client. ZodError retourne un message métier explicite.

8.6 Limitation documentée

Rate limiter en mémoire non partagé entre instances Vercel. Migration Upstash Redis planifiée V2.

9. Actions pour l'accessibilité

9.1 Référentiel retenu

WCAG 2.1 niveau AA (W3C), aligné sur les exigences RGAA 4.1 pour les services numériques en France. La checklist OPQUAST (bonnes pratiques qualité web) guide les revues de contenu et de formulaires.

9.2 Stratégie technique

Radix UI / shadcn : primitives conformes WAI-ARIA 1.1 - focus trap dans les dialogues, navigation clavier (Tab, Escape, flèches), attributs role, aria-expanded, aria-selected générés automatiquement.

15 composants utilisent explicitement aria-* ou sr-only : dialog, select, sheet, carousel, tables admin, kitchen-status-control, mode-toggle, etc.

9.3 Critères WCAG implémentés

Critère	Niveau	Implémentation
1.1.1 Non-text Content	A	alt sur images informatives
1.3.1 Info and Relationships	A	Balises sémantiques (nav, main, header, footer)
1.4.3 Contrast Minimum	AA	Palette Tailwind, ratio $\geq 4,5:1$
2.1.1 Keyboard	A	Composants Radix navigables au clavier
2.4.3 Focus Order	A	Ordre de tabulation logique
2.4.7 Focus Visible	AA	focus-visible:ring sur boutons shadcn
3.1.1 Language of Page	A	lang="fr" sur les layouts
3.3.1 Error Identification	A	Messages liés via aria-describedby
4.1.2 Name, Role, Value	A	ARIA natif Radix

9.4 Contrôles effectués

- Audit Lighthouse en développement (score accessibilité cible > 90) - voir Annexe G
- Tests clavier manuels sur dashboard et formulaires admin
- Labels associés aux champs (htmlFor / id) sur tous les formulaires

10. Historique des versions

10.1 Versioning applicatif

Version	Date	Jalons
0.1.0	Q1 2026	MVP : dashboard, commandes, menu manuel
0.1.5	Avr 2026	Intégration agent vocal, webhooks, HubRise
0.2.0	Mai-Juin 2026	Migration Vapi, organisations multi-tenant, rôles EMPLOYEE, call_logs, CI/CD complet

Version actuelle dans package.json : **0.2.0**.

10.2 Stratégie Git

- Branches : feature/* -> develop -> main
- Commits : Conventional Commits (feat:, fix:, test:, chore:)
- PR avec revue avant merge
- Chaque déploiement Vercel lié à un commit SHA

10.3 Évolution du schéma (migrations Drizzle)

42 fichiers SQL dans drizzle/. Jalons significatifs :

Migration	Contenu
0000-0003	Tables users, restaurants, orders, order_items
0011	Statut cuisine (kitchenStatus, statusSettings JSONB)
0012	Menu en JSONB (menuData) - abandon tables menu relationnelles
0022	Champs abonnement Stripe
0023	Migration Supabase Auth (suppression colonne password)
0027-0028	Agent Vapi, message d'accueil, organisations
0029	Table call_logs (traçabilité appels)
0030-0031	Stripe au niveau organisation, statut org
0032-0033	Rôles OWNER / EMPLOYEE / restaurant_members

10.4 Évolution de l'agent vocal

Phase	Technologie	Statut
V1 initiale	ElevenLabs ConvAI	Remplacé
V1 actuelle	Vapi	Production (vapiAssistantId, /api/vapi/webhook)
Legacy	Route /api/elevenlabs/webhook	Conservée pour rétrocompatibilité

11. Présentation de la dernière version

11.1 État fonctionnel

La version 0.2.0 déployée sur app.yallo.fr est fonctionnelle, fiable et manipulable en autonomie par un utilisateur formé.

Parcours complet opérationnel :

- 1. L'admin crée organisation, restaurant et utilisateur
- 2. L'admin configure l'agent Vapi (createVapiAgent) avec numéro Twilio
- 3. Le restaurateur configure menu et horaires
- 4. Le client appelle -> agent vocal prend commande -> dashboard affiche la commande
- 5. Le cuisinier fait progresser la commande -> SMS de confirmation envoyé
- 6. L'abonnement est géré via Stripe (webhook synchronisation)

11.2 Modules livrés

Module	Route	Fonctionnalités
Marketing	yallo.fr	Accueil, démo, contact, pages légales
Dashboard	/dashboard	Commandes temps réel, statut cuisine, menu, horaires
Organisation	/org	Gestion multi-restaurants (OWNER)
Admin	/admin	CRUD restaurants, users, agents Vapi
Facturation	/org/billing	Abonnement Stripe, usage
Agent vocal	Webhook Vapi	Prise commande, validation horaires/STOP

11.3 Fiabilité

- Pipeline CI bloquant sur tests et build
- Monitoring Sentry en production (erreurs anonymisées)
- Webhooks idempotents (onConflictDoNothing sur call_logs)
- Rollback agent Vapi si import Twilio échoue

11.4 Autonomie utilisateur

Un restaurateur peut, après onboarding admin :

- Se connecter sur app.yallo.fr
- Configurer son menu (saisie ou import photo GPT-4o)
- Définir ses horaires et son statut cuisine
- Traiter ses commandes sans assistance technique

12. Cahier de recettes

12.1 Méthodologie

Recette en trois niveaux : fonctionnel (parcours utilisateur), technique (API, webhooks), sécurité (contrôle d'accès, injection). Chaque scénario précise prérequis, étapes, résultat attendu et statut. Tous les scénarios ont été joués localement, sur l'environnement Staging et aussi en Production.

12.2 Recette fonctionnelle

ID	Module	Scénario	Résultat attendu	Statut
RF-01	Auth	Connexion OWNER valide	Redirection /org, session active	OK
RF-02	Auth	Connexion ADMIN valide	Redirection /admin	OK
RF-03	Auth	Connexion EMPLOYEE valide	Redirection /dashboard	OK
RF-04	Auth	Accès /admin sans rôle ADMIN	Redirection /org ou /dashboard	OK
RF-05	Commandes	Affichage nouvelles commandes	Liste à jour après webhook Vapi	OK
RF-06	Commandes	Changement statut Nouvelle -> Prête	Statut mis à jour, UI rafraîchie	OK
RF-07	Cuisine	Passage statut STOP	Agent refuse nouvelle commande	OK
RF-08	Cuisine	Retour statut NORMAL	Agent accepte commandes	OK
RF-09	Menu	Sauvegarde menu JSONB	Menu persisté, agent Vapi synchronisé	OK
RF-10	Menu	Import 3 photos menu	Structure MenuData générée par GPT-4o	OK
RF-11	Horaires	Horaires hors plage	Agent refuse commande hors horaires	OK
RF-12	Admin	Création restaurant + agent Vapi	vapiAssistantId renseigné en base	OK
RF-13	Facturation	Webhook Stripe subscription.updated	Statut org synchronisé	OK
RF-14	SMS	Commande réussie	SMS confirmation envoyé (si configuré)	OK
RF-15	HubRise	Commande avec HubRise actif	Push commande vers caisse (non bloquant)	OK

12.3 Recette technique

ID	Scénario	Résultat attendu	Statut
RT-01	pnpm run test:run	0 échec	OK
RT-02	pnpm run build	Build sans erreur TypeScript	OK
RT-03	Pipeline CI sur main	6 jobs success	OK
RT-04	Webhook Vapi sans secret (prod)	HTTP 401	OK
RT-05	Webhook Vapi payload invalide	HTTP 400, pas de commande créée	OK
RT-06	Webhook Stripe signature invalide	HTTP 400	OK
RT-07	Bootstrap base vierge via "pnpm db:push"	Schéma complet appliqué	OK

ID	Scénario	Résultat attendu	Statut
RT-08	Middleware domaine marketing	Routes /dashboard redirigées vers app.	OK

12.4 Recette sécurité

ID	Scénario	Résultat attendu	Statut
RS-01	Server Action sans session	Erreur « Non autorisé »	OK
RS-02	EMPLOYEE accède restaurant non assigné	Accès refusé	OK
RS-03	Injection SQL via champ menu	Requête paramétrée, pas d'injection	OK
RS-04	Rate limit dépassé (11 req/min)	Requête refusée	OK
RS-05	En-têtes HTTP sécurité	HSTS, X-Frame-Options présents	OK
RS-06	Sentry	sendDefaultPii: false, pas de PII	OK
RS-07	Webhook Vapi secret incorrect	HTTP 401	OK
RS-08	OWNER tente /admin	Redirection /org	OK

12.5 Recette structurelle

ID	Scénario	Résultat attendu	Statut
RSt-01	Build production	0 erreur	OK
RSt-02	SonarCloud Quality Gate	Passed	OK
RSt-03	Couverture ≥ 70 %	72,55 %	OK
RSt-04	TypeScript strict	0 erreur tsc	OK
RSt-05	Déploiement zero-downtime	Bascule Vercel OK	OK

12.6 Synthèse

36 scénarios exécutés, 36 réussis (100 %). Couvre l'ensemble des modules : auth, commandes, cuisine, menu, horaires, admin, facturation, webhooks, sécurité.

13. Plan de correction des bogues

13.1 Méthodologie

Chaque anomalie est qualifiée (Bloquant / Majeur / Mineur), analysée (cause racine), corrigée, puis couverte par un test de non-régression.

13.2 Anomalies tracées - série webhook agent vocal

ID	Sévérité	Impact	Correction	Statut
BUG-01	Bloquant	Commandes non enregistrées	Ajout ?rid= dans URL webhook Vapi	OK
BUG-02	Majeur	Erreurs silencieuses	Logger avec contexte (restaurantId, payload)	OK
BUG-03	Majeur	Payload non reconnu	Normalisation via normalizeSubmitOrderPayload	OK
BUG-04	Mineur	Mode strict en local	Strict uniquement si VERCEL_ENV === production	OK
BUG-05	Majeur	Commande sans items	Validation items.length > 0 avant insertion	OK
BUG-06	Mineur	Menu non sync agent	Appel sync dans saveMenuData	OK
BUG-07	Majeur	Agent orphelin si Twilio échoue	Suppression agent Vapi si import Twilio échoue	OK

13.3 Analyse des échecs de recette

Pour chaque test en échec lors de la recette :

- 1. Reproduction locale documentée
- 2. Identification cause (code, config, service tiers)
- 3. Correctif ciblé + test unitaire
- 4. Re-exécution recette complète du module impacté

14. Manuel de déploiement

14.1 Prérequis

- Node.js 20+, pnpm 9+
- Comptes : GitHub, Vercel, Supabase, Stripe, Vapi, Twilio
- Accès aux secrets d'environnement production

14.2 Installation locale

```
git clone <repo>
cd yallo
pnpm install
cp .env.example .env.local # renseigner les variables
pnpm drizzle-kit migrate    # appliquer migrations
pnpm dev
```

URLs locales : marketing <http://localhost:3000>, app <http://app.localhost:3000>.

14.3 Variables d'environnement critiques

Variable	Description
DATABASE_URL	PostgreSQL Supabase
NEXT_PUBLIC_SUPABASE_URL / ANON_KEY	Auth client
SUPABASE_SERVICE_ROLE_KEY	Auth serveur
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	Paieement
VAPI_API_KEY / VAPI_WEBHOOK_SECRET	Agent vocal
TWILIO_ACCOUNT_SID / AUTH_TOKEN	Téléphonie / SMS
OPENAI_API_KEY	Parsing menu
SENTRY_DSN	Monitoring
NEXT_PUBLIC_APP_URL	URL canonique app

14.4 Déploiement automatique (recommandé)

1. Merger sur develop -> staging déployé automatiquement
2. Valider sur staging.yallo.fr
3. Merger sur main -> production déployée via ci.yml

14.5 Déploiement manuel (secours)

```
pnpm add -g vercel
vercel --env NEXT_PUBLIC_APP_URL=https://staging.yallo.fr # staging
vercel --prod                                             # production
```

14.6 Migrations en production

1. Tester migration sur base staging
2. Appliquer : pnpm drizzle-kit migrate avec DATABASE_URL production
3. Vérifier logs Vercel et Sentry dans les 10 minutes suivantes

14.7 Rollback

- **Code** : Vercel Dashboard -> Promote previous deployment (instantané)
- **Base** : migrations Drizzle sont forward-only ; prévoir scripts de rollback manuels si nécessaire

15. Manuel d'utilisation

15.1 Profils utilisateur

Profil	Rôle	Interface principale
Administrateur Yallo	ADMIN	/admin
Restaurateur / gérant	OWNER	/org, /dashboard
Employé cuisine	EMPLOYEE	/dashboard

15.2 Restaurateur (OWNER) - Première configuration

- 1. Se connecter sur app.yallo.fr avec identifiants reçus
- 2. Accéder à Menu (/dashboard/menu) : saisir ou importer photos de la carte
- 3. Vérifier et enregistrer le menu généré
- 4. Accéder à Horaires (/dashboard/hours) : configurer plages d'ouverture
- 5. Vérifier statut cuisine sur NORMAL
- 6. Tester : appeler le numéro dédié, passer une commande test
- 7. Vérifier apparition de la commande (statut Nouvelle) sur le dashboard

15.3 Employé cuisine (EMPLOYEE) - Usage quotidien

- 1. Se connecter -> redirection automatique /dashboard
- 2. Consulter les commandes entrantes
- 3. Faire progresser chaque commande : Nouvelle -> En préparation -> Prête -> Livrée
- 4. Adapter le statut cuisine selon la charge : Calme / Normal / Rush / Stop (refus automatique des appels)

15.4 Administrateur (ADMIN) - Onboarding restaurant

- 1. Se connecter -> redirection /admin
- 2. Créer l'utilisateur restaurateur (/admin/users/new)
- 3. Créer le restaurant (/admin/restaurants/new), associer l'owner
- 4. Renseigner le numéro Twilio E.164
- 5. Cliquer Créer l'agent Vapi - attendre confirmation (10-15 s)
- 6. Vérifier vapiAssistantId et vapiPhoneNumberId en base
- 7. Appel test sur le numéro Twilio

15.5 Exploitant technique

- 1. Surveiller Sentry (erreurs production)
- 2. Contrôler runs GitHub Actions
- 3. Vérifier webhooks Stripe et Vapi (logs Vercel)
- 4. Appliquer migrations si évolution schéma

16. Manuel de mise à jour

16.1 Processus standard

- 1. git pull origin develop (ou main)
- 2. pnpm install (met à jour dépendances selon lockfile)
- 3. Consulter nouvelles migrations dans drizzle/
- 4. Appliquer migrations sur staging : pnpm drizzle-kit migrate
- 5. Valider fonctionnellement sur staging
- 6. pnpm run lint && pnpm run test:run && pnpm run build
- 7. Merger vers main si validation OK -> déploiement production automatique
- 8. Appliquer migrations sur base production
- 9. Surveiller Sentry 10 minutes post-déploiement

16.2 Mise à jour des dépendances

```
pnpm update <package>      # mise à jour ciblée
pnpm audit                  # vérifier vulnérabilités
pnpm run test:run && pnpm build # valider non-régression
```

16.3 Évolution du schéma

- 1. Modifier src/db/schema.ts
- 2. pnpm drizzle-kit generate -> nouveau fichier SQL
- 3. Tester sur staging
- 4. Commit + merge -> appliquer en production après déploiement

16.4 Principes de sécurité des mises à jour

- Jamais déployer en production sans validation staging
- Migrations backward-compatible (ajout colonnes, pas de rename destructif en une étape)
- Variables d'environnement nouvelles : configurer dans Vercel avant le déploiement

Conclusion

Le projet Yallo démontre la maîtrise complète des compétences du Bloc 2 RNCP 39583 : environnements industrialisés (CI/CD Vercel + GitHub Actions), architecture maintenable (feature-first, TypeScript strict, Drizzle), tests unitaires avec couverture > 70 %, sécurité OWASP, accessibilité WCAG 2.1 AA, recette structurée et documentation d'exploitation.

Les axes d'amélioration V2 identifiés : rate limiting distribué (Upstash Redis), tests E2E Playwright, documentation OpenAPI des webhooks, CHANGELOG versionné.

Annexes

Annexe	Contenu
A	Captures site marketing
B	Captures dashboard production
C	Rapport couverture Vitest (pnpm test:coverage)
D	Capture SonarCloud - Quality Gate Passed
E	Capture GitHub Actions - run ci.yml
F	Audit Lighthouse - scores Performance et Accessibilité
G	Schéma ERD base de données
H	Liste complète des 42 migrations Drizzle

Annexe A :

Simple comme bonjour

Comment ça marche ?

1 ÉTAPE 1

Fini la tonalité 'Occupé'

1 client ou plusieurs appellent en même temps ? Yallo gère des appels en simultané. Ne perdez plus jamais une commande parce que vous étiez déjà au téléphone.

Appels simultanés Réponse immédiate Zéro attente

Appels entrants 2 Live

06 12 34 56 78 Client régulier

06 98 76 54 32 Nouveau client

Yallo gère tout simultanément...

2 ÉTAPE 2

L'IA prend la commande

Notre IA comprend le client, gère les demandes spéciales et suggère intelligemment des extras si vous le souhaitez. Elle parle naturellement, sans accent robotique.

Compréhension vocale Upsell intelligent Multilingue

3 ÉTAPE 3

Transmission instantanée en cuisine

Dès que le client raccroche, le bon de commande arrive sur votre dashboard ou bien votre logiciel de caisse. Zéro ressaisie, zéro erreur. Vos chefs lancent la cuisson immédiatement.

Commandes automatiques Zéro ressaisie Prêt à cuisiner

COMMANDE #2847
Pour 1645 - Tél: 06 12 34 56 78

1x Menu Classique	9.50€
+ Supplément fromage	
1x Accompagnement	3.00€
1x Coca-Cola	2.50€
TOTAL	14.00€

Fonctionnalités

Tout ce dont vous avez besoin



Compréhension Vocale Avancée

L'IA comprend les accents régionaux, le vocabulaire de chaque cuisine et les demandes complexes, dans n'importe quel contexte.



Accents Slang Multilingue 24/7



Multi ligne

Plusieurs appels simultanés. Jamais de sonnerie occupée.



Appel 1
Actif



Appel 2
Actif



Appel 3
Actif



Appel 4
Actif



Import de Carte

Importez votre carte PDF, l'IA génère votre menu automatiquement.

carte-restaurant.pdf

Menu généré par l'IA



Upsell Intelligent

L'IA suggère naturellement boissons et accompagnements selon la commande. Sans jamais forcer.



+18%
Panier moyen



Relais Humain

L'IA transfère l'appel à votre équipe pour les demandes qui sortent du standard. Aucune perte de contact.



Yallo IA
En ligne



Votre équipe
Disponible



Intégration Caisse

Connecté à HubRise, menu et stocks synchronisés en temps réel.



Yallo



HubRise synchronisé



Caisse



Un prix simple et transparent

Trois offres claires, pensées pour votre volume d'appels.

Essentiel

Pour les petits établissements qui veulent tester sans risque

49€ /mois
+ 0,09€ / minute d'appel

- ✓ 1 numéro de téléphone dédié
- ✓ 2 appels simultanés max
- ✓ IA vocale 24h/24, 7j/7
- ✓ Dashboard tablette
- ✓ Gestion des commandes en temps réel
- ✓ Menu illimité
- ✓ Mise à jour instantanée
- ✓ Support email

Commencer →

Pro

Le plus choisi

Pour les établissements actifs qui veulent une solution complète

99€ /mois
+ 0,07€ / minute d'appel

TOUT PAREIL QUE ESSENTIEL +

- ✓ 4 appels simultanés max
- ✓ Connexion logiciel de caisse
- ✓ Sépare commandes & renseignements
- ✓ Analytics & historique des appels
- ✓ Rapports hebdomadaires par email
- ✓ Support email prioritaire
- ✓ Réponse sous 4h en semaine

Démarrer →

Business

Pour les établissements à fort volume ou les chaînes multi-sites

199€ /mois
+ 0,05€ / minute d'appel

TOUT PAREIL QUE PRO +

- ✓ 1 numéro de téléphone dédié par restaurant
- ✓ 10 appels simultanés max
- ✓ Support prioritaire
- ✓ Transfert vers humain
- ✓ Bascule vers un employé si besoin
- ✓ Personnalisation avancée de l'IA
- ✓ Ton, script, promotions...
- ✓ Multi-sites
- ✓ Gérez plusieurs adresses depuis un dashboard
- ✓ Support dédié
- ✓ Interlocuteur fixe, réponse sous 1h

Commencer →

Questions Fréquentes

Tout ce que vous devez savoir sur Yallo et comment nous pouvons transformer votre prise de commande.

Une autre question ?

Notre équipe vous répond sous 24h.

support@yallo.com →

Comment fonctionne Yallo concrètement ?

L'IA comprend-elle bien les clients au téléphone ?

Combien de temps prend la mise en place ?

Puis-je transférer un appel à un employé ?

Annexe B :

Dashboard Admin

Gérez vos organisations, restaurants et utilisateurs

Organisations

Gérez les organisations et leurs abonnements

+ Nouvelle organisation

1

Total

1

Actives

0

Onboarding

0

Suspendues

0

Abonnées Stripe

Q

Rechercher par nom...

Q

Tous les statuts

Organisation	Membres	Statut	Restaurants	Stripe	Créée le	
Burger King	copeaux.lenny@gmail.com	Actif	2	—	19/07/2026	...

Yallo

Admin

Dashboard

Rôles

←

Burger King Saint André de Cubzac

Onboarding

copeaux.lenny@gmail.com

IA Non configurée

Général

IA & Menu

Téléphonie

HubRise

Utilisation

Informations générales

Modifiez les informations de base du restaurant

Nom du restaurant *

Burger King Saint André de Cubzac

Statut *

Onboarding

Organisation

Burger King

Membres

1 membre

Adresse

87 rue de Saint andre

Créé le 19 juillet 2026

Modifié le 19 juillet 2026 à 19:26

Enregistrer

Annexe C :

Test Files

29 passed (29)

Tests

215 passed (215)

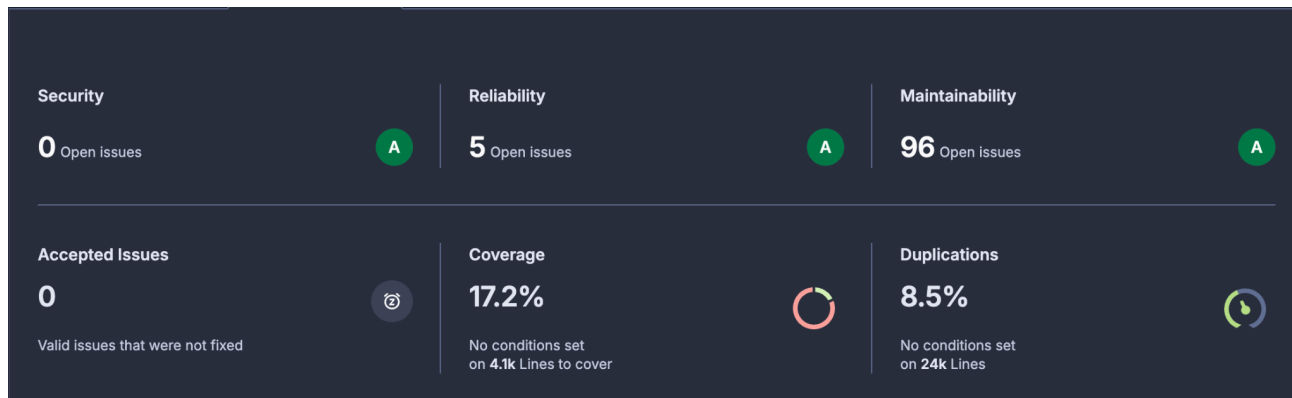
Start at

20:41:52

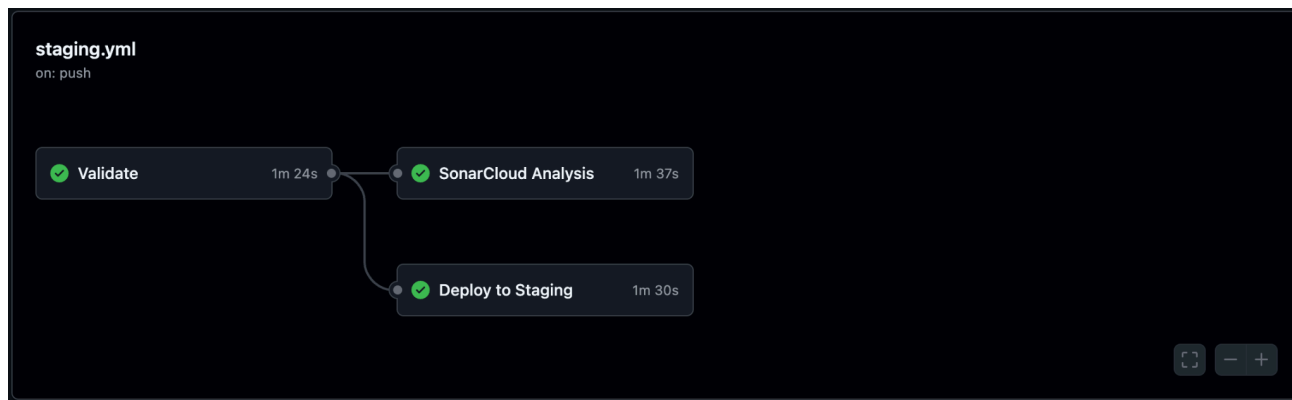
Duration

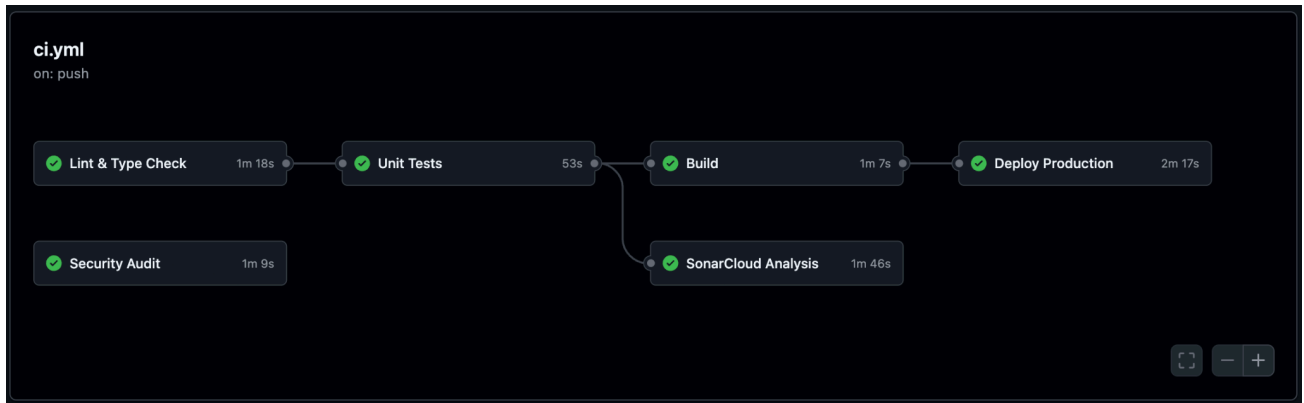
10.14s (transform 223ms, setup 1.31s, import 1.36s, tests 321ms, environment 4.09s)

Annexe D :

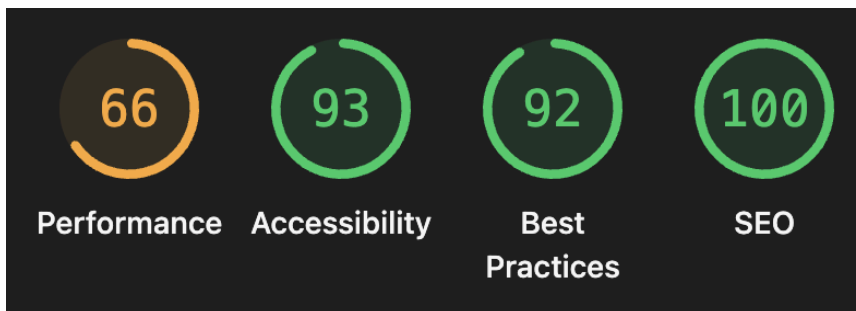


Annexe E :

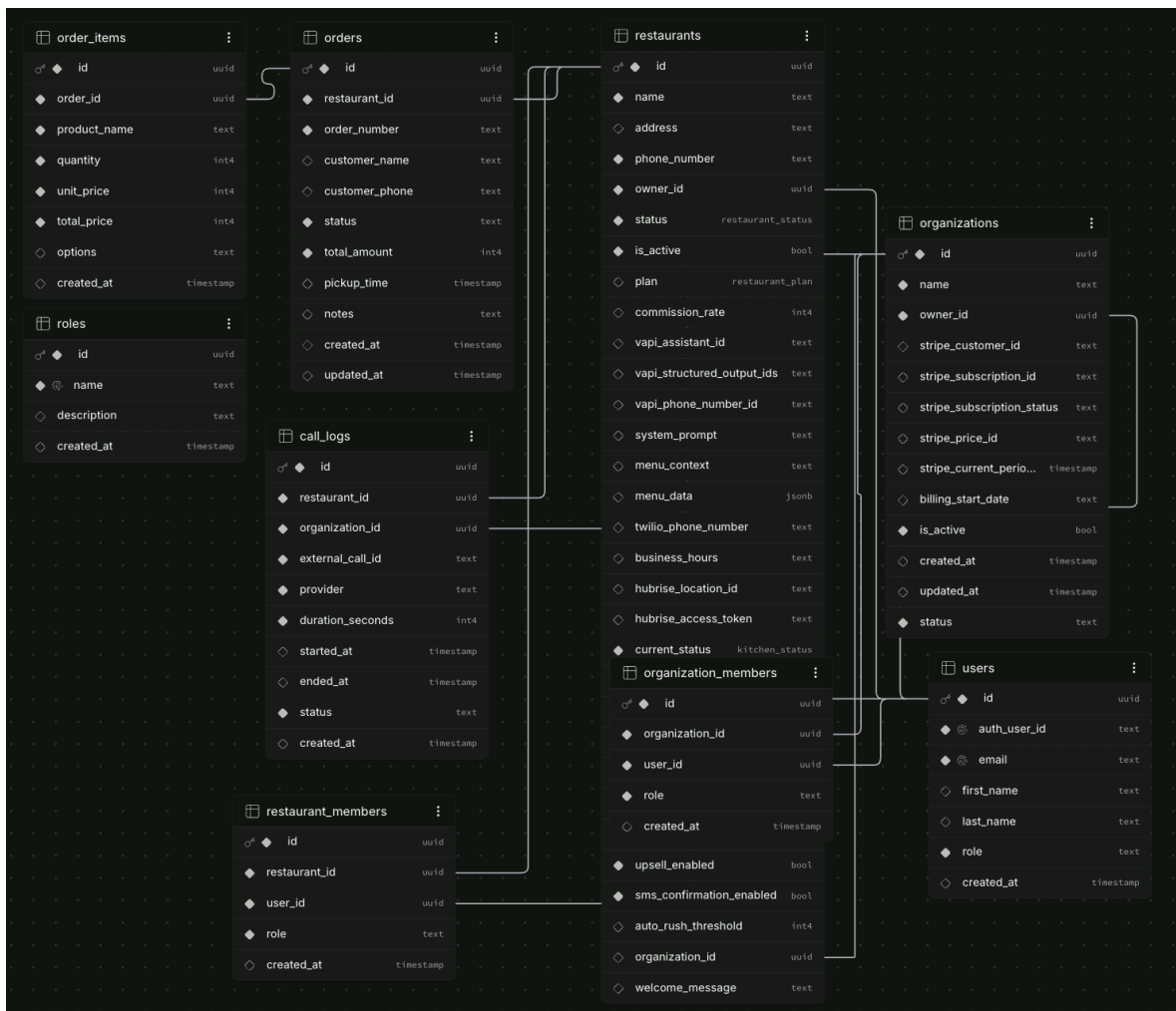




Annexe F :



Annexe G :



Annexe H :

0000_gifted_guardian.sql	0019_drop_pricing_plans.sql
0001_worried_lockheed.sql	0019_dry_apocalypse.sql
0002_clever_pepper_potts.sql	0020_add_vapi_structured_output_ids.sql
0003_orders_tables.sql	0021_add_hubrise_catalog_id.sql
0004_admin_crm_restaurants.sql	0022_add_stripe_subscription_fields.sql
0005_add_billing_start_date.sql	0023_add_auth_user_id_remove_password.sql
0006_add_user_names.sql	0024_add_elevenlabs_agent.sql
0007_add_pricing_config.sql	0024_remove_legacy_fields.sql
0008_dizzy_mantis.sql	0025_add_call_forwarding.sql
0009_add_is_available_to_variations.sql	0025_drop_hubrise_catalog_id.sql
0010_remove_image_url_from_ingredients.sql	0026_add_assistant_settings.sql
0011_add_kitchen_status.sql	0027_add_members_tables.sql
0011_serious_sheva_callister.sql	0027_add_welcome_message.sql
0012_remove-menu-tables-add-menudata.sql	0027_rename_voice_id.sql
0013_remove-hubrise-catalog-cache.sql	0028_add_organizations.sql
0014_remove-forwarding-phone-number.sql	0029_add_call_logs.sql
0015_loud_molecule_man.sql	0030_remove_stripe_from_restaurants.sql
0015_remove-slug.sql	0031_add_org_status.sql
0016_add_pricing_plans.sql	0032_add_roles.sql
0016_melodic_sunset_bain.sql	0033_simplify_roles.sql
0017_remove_pricing_config.sql	
0018_add_vapi_phone_number_id.sql	